

GIÁO TRÌNH SBA301 – SLOT 12

CHAPTER 10: RESTFUL WEB SERVICES WITH SPRING BOOT

PART A – REST FUNDAMENTALS

Resource • URI • HTTP Methods • Status Codes • REST Constraints • Spring Boot Overview

CLO3 | JDK 21 | Spring Boot 3 (baseline môn học) | Project-Based Learning

Phạm vi slide chuẩn: Chapter 10, trang 1–17

Thông tin nhanh

Thuộc tính	Nội dung
Slot	12 – buổi đầu của Chapter 10
Chủ đề	RESTful Web Services with Spring Boot – Part A: REST fundamentals
Mục tiêu	Từ tư duy frontend client → thiết kế backend API contract đúng REST
Không đi sâu	Spring annotations, controller/service/repository, 3-layer CRUD, exception handling chi tiết
Sản phẩm học tập	REST design worksheet + API contract FUNews + Postman evidence + Spring Boot skeleton
Buổi kế tiếp	Part B – annotations + 3-layer CRUD + request/response handling

Ranh giới nội dung — Part A ưu tiên “design before annotations”. Mọi ví dụ annotation chỉ là preview để sinh viên hình dung. Không triển khai CRUD Spring Boot hoàn chỉnh trong tài liệu này.

Cách sử dụng giáo trình

- Giảng viên: dẫn dắt bằng mental model, REST design workshop, Postman observation và case study FUNewsManagementSystem.
- Sinh viên: trước khi code backend phải xác định resource → URI → method → status → representation → API contract.
- Mỗi khối kỹ thuật đi theo nhịp: Mục tiêu → Khái niệm → Ví dụ → Giải thích → Lỗi → Best practice → Bài tập nhỏ.
- AI có thể gợi ý API design nhưng sinh viên phải kiểm chứng bằng REST rules, HTTP semantics và yêu cầu nghiệp vụ.

MỤC LỤC NỘI DUNG

- 0. Sinh viên cần chuẩn bị gì trước khi học Slot 12
- I. Tóm tắt các chủ đề & mục tiêu trong bài học
- II. Trọng tâm cốt lõi & kỹ năng sinh viên cần nắm vững
- III. Ứng dụng thực tiễn trong sản phẩm thật
- IV. Hướng dẫn thực hiện và triển khai Part A
- V. Kinh nghiệm thực tiễn, lỗi và tips
- VI. Bài tập chủ đề & bài tập tổng hợp FUNews API Design
- VII. Foundation Glossary
- VIII. Research Base & AI-Based Learning
- IX. Tự kiểm tra sau buổi học

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT 12

Slot 12 đánh dấu chuyển pha từ frontend ReactJS sang backend Spring Boot. Sinh viên không cần viết REST controller hoàn chỉnh trong buổi này, nhưng phải mang theo mental model HTTP/JSON/REST đã học và khả năng quan sát request/response.

0.1 Kiến thức cần ôn lại

Nguồn kiến thức	Cần nhớ	Câu nối sang Slot 12
Slot 8	Client–Server, request/response, HTTP methods, status, JSON, REST	Hiểu giao tiếp API ở mức giao thức
Slot 9	Promise, async/await, Fetch/Axios, loading/error/data	Biết client gọi API như thế nào
Slot 10	SPA/React Router	Biết route phía client khác API endpoint
Lab 02	React SPA, component, service/fetch	Thấy frontend cần một data provider ổn định
Assignment 01	FUNews frontend: News/Category/User CRUD/Search	Dùng chính domain này để thiết kế backend API contract

0.2 Câu hỏi nền tảng phải trả lời được

- Client gửi request cho ai và server trả lại gì?
- GET/POST/PUT/DELETE khác nhau ở mục đích nào?
- 200, 201, 400, 401, 403, 404, 500 biểu thị điều gì?
- JSON là representation hay protocol?
- React route /news/10 và API endpoint /api/news/10 khác nhau ở đâu?

0.3 Công cụ cần chuẩn bị

- JDK 21 và JAVA_HOME hoạt động đúng.
- IntelliJ IDEA; Maven/Gradle theo project.
- Postman; Chrome/Edge DevTools.
- Spring Initializr để tạo project skeleton.
- Git/GitHub nếu lớp yêu cầu evidence theo issue/branch/commit/PR.

Baseline môi trường — Giáo trình giữ baseline môn học JDK 21 + Spring Boot 3. Khi Spring Initializr hiển thị phiên bản mới hơn, sinh viên chọn phiên bản Spring Boot 3.x được giảng viên/course baseline phê duyệt thay vì tự ý đổi nền tảng.

0.4 Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn giải đáp
HTTP/JSON/REST...	Stateless? URI? sub-resource?	Ví dụ: Vì sao /api/getNews không phải thiết kế tốt?

0.5 Pre-lab quan sát một API thật

Dùng Postman gọi GET <https://jsonplaceholder.typicode.com/posts> và ghi lại method, URL, status code, Content-Type, kiểu response, resource mà endpoint đại diện. Không cần viết code.

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

I.1 Bảng tóm tắt giai đoạn trước

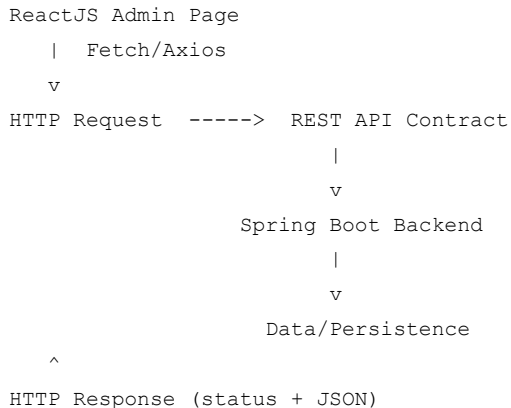
Giai đoạn	Chủ đề	Sinh viên đã làm được	Vấn đề Slot 12 giải quyết
Slot 8	HTTP, JSON, REST	Đọc request/response, mapping CRUD	REST mới ở mức client/API concept
Slot 9	Fetch/Axios	Gọi API từ JavaScript/React	Chưa tự xây backend API
Slot 10	SPA/Router	Điều hướng nhiều view	Route không cung cấp dữ liệu
Lab 02	Orchid SPA	UI + service/fetch	Data source vẫn mock/public JSON
Assignment 01	FUNews frontend	CRUD/search phía frontend	Cần backend chuẩn cho News/Category/User

I.2 Định vị Slot 12 trong lộ trình backend

Buổi/Giai đoạn	Chủ đề	Quan hệ với Part A
Slot 12	REST fundamentals	Thiết kế resource, URI, methods, status, constraints, API contract
Part B	Annotations + 3-layer CRUD	Biến API contract thành controller/service/repository
Lab 03	RESTful Web Services with Spring Boot	Thực hành REST API
Chapter 11	Documenting REST Services	OpenAPI/Swagger dựa trên contract đã thiết kế
Chapter 12	Versioning/paging/sorting/testing	Nâng API từ CRUD cơ bản lên production concerns
Chapter 13	REST + JPA	Kết nối resource với persistence
Chapter 14/15	React + Spring Boot	Frontend Assignment/Lab gọi backend thật

I.3 Bối cảnh dẫn nhập: từ FUNews frontend sang backend

MENTAL MODEL



Ở Assignment 01, frontend có News, Category, Users và Search. Khi dữ liệu không còn nằm trong state/localStorage, frontend cần một backend cung cấp API nhất quán. Slot 12 tập trung vào cách mô tả “hợp đồng” đó trước khi viết controller.

I.4 Mục tiêu sau buổi học

- Giải thích REST và RESTful Web Services bằng ngôn ngữ của chính mình.
- Xác định resource, collection URI, item URI và sub-resource.
- Mapping CRUD sang GET/POST/PUT/DELETE ở mức tổng quan.
- Chọn status code cơ bản theo kết quả request.
- Giải thích Client–Server, Stateless, Cacheable, Uniform Interface, Layered System.
- Thiết kế API contract cơ bản cho FUNewsManagementSystem.
- Giải thích vai trò của Spring Boot/Spring Web trước khi học annotations.
- Tạo được Spring Boot skeleton bằng Spring Initializr nhưng chưa triển khai CRUD.

I.5 Khái niệm trung tâm

Khái niệm	Một câu phải nhớ
REST	Phong cách kiến trúc, không phải framework.
Resource	Danh từ/thực thể mà API expose.
URI	Định danh resource.
Representation	Cách biểu diễn trạng thái resource, thường JSON.
HTTP method	Ý định thao tác trên resource.
Status code	Kết quả của request.
Stateless	Mỗi request mang đủ context cần thiết; server không dựa vào client-session state giữa các request.
API contract	Thỏa thuận method + endpoint + request + response + status.
Spring Boot	Nền tảng triển khai ứng dụng Spring nhanh với auto-configuration/starter/embedded server.

I.6 Flow bài học

LEARNING FLOW

```

Frontend đã biết gọi API
    ↓
Backend phải expose resource
    ↓
Resource → URI
    ↓
Method → hành động
    ↓
Status → kết quả
    ↓
Representation → JSON/XML
    ↓
REST constraints → kiến trúc nhất quán
    ↓
API contract → Spring Boot skeleton
    ↓
Part B: annotations + 3-layer CRUD
  
```

I.7 Tình huống dẫn nhập

Nhiệm vụ FUNews	API dự kiến	Điều sinh viên phải giải thích
Xem danh sách News	GET /api/news	Vì sao resource dùng danh từ?
Xem News #10	GET /api/news/10	Collection URI và item URI khác nhau thế nào?
Tạo News	POST /api/news	Vì sao không dùng /api/createNews?
Sửa News #10	PUT /api/news/10	Vì sao id nằm trong URI?
Xóa News #10	DELETE /api/news/10	Status nào thể hiện thành công?

I.8 Tư duy kiến trúc quan trọng

- REST không đồng nghĩa với Spring Boot. Spring Boot là công cụ triển khai; REST là style thiết kế.
- Client route và API endpoint phục vụ hai mục đích khác nhau.
- Thiết kế API phải bắt đầu từ domain/resource, không bắt đầu từ tên hàm controller.
- HTTP method truyền đạt intent; URI truyền đạt resource.
- Status code và error representation là một phần của contract, không phải chi tiết trang trí.

I.9 Tài liệu tham khảo trong phạm vi buổi học

- Slide Chapter 10, trang 1–17.
- Spring REST Tutorial – spring.io/guides/tutorials/rest/.
- Building a RESTful Web Service – spring.io/guides/gs/rest-service/.
- Baeldung REST with Spring Series (tham khảo mở rộng).
- Tài liệu Slot 8, Slot 9, Lab 02 và Assignment 01.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

II.1 REST là gì?

Mục tiêu: Hiểu REST là architectural style cho distributed/networked systems.

Ví dụ: React client giao tiếp với Spring Boot server thông qua resource-oriented API.

Giải thích: REST định hướng cách tách client/server, định danh resource và sử dụng giao diện thống nhất.

Lỗi/hiểu sai thường gặp: Gọi REST là framework hoặc “một thư viện Java”.

Best practice: Mô tả REST bằng constraints và resource interaction, không bằng tên công nghệ.

Tự kiểm tra: Nói REST trong 30 giây mà không dùng từ Spring Boot.

II.2 RESTful Web Services

Mục tiêu: Nhận biết một web service được thiết kế theo REST.

Ví dụ: GET /api/news trả representation của collection News.

Giải thích: Web service dùng HTTP, URI và representations để thao tác resource.

Lỗi/hiểu sai thường gặp: Cho rằng REST bắt buộc chỉ được dùng JSON.

Best practice: JSON phổ biến nhưng representation không đồng nhất với REST.

Tự kiểm tra: Liệt kê resource/method/representation của một API quen thuộc.

II.3 Resource và Representation

Mục tiêu: Phân biệt resource với dữ liệu biểu diễn resource.

Ví dụ: Resource News #10; representation JSON gồm id/title/status/category.

Giải thích: Resource là khái niệm domain; JSON chỉ là trạng thái được biểu diễn để truyền qua mạng.

Lỗi/hiểu sai thường gặp: Nghĩ JSON object chính là resource.

Best practice: Thiết kế resource theo domain nouns; representation theo nhu cầu client.

Tự kiểm tra: Với User, chỉ ra field nào có thể không nên expose.

II.4 URI và endpoint

Mục tiêu: Thiết kế collection/item URI nhất quán.

Ví dụ: /api/news và /api/news/10.

Giải thích: URI xác định resource; endpoint thực tế kết hợp method + URI để tạo interaction.

Lỗi/hiểu sai thường gặp: Đặt URI dạng /getAllNews hoặc /deleteNews.

Best practice: Dùng nouns, số nhiều cho collection, cấu trúc nhất quán.

Tự kiểm tra: Sửa 5 endpoint verb-based thành resource-based.

II.5 HTTP methods tổng quan

Mục tiêu: Mapping CRUD với GET/POST/PUT/DELETE.

Ví dụ: POST /api/categories; PUT /api/categories/5.

Giải thích: Method biểu thị intent, giúp client/server hiểu semantics mà không nhét động từ vào URI.

Lỗi/hiểu sai thường gặp: Dùng POST cho mọi thao tác.

Best practice: Dùng method đúng semantics; Part B sẽ học request/body/annotation chi tiết.

Tự kiểm tra: Chọn method cho 8 thao tác FUNews.

II.6 HTTP status codes

Mục tiêu: Chọn status code cơ bản theo outcome.

Ví dụ: 201 khi tạo News; 404 khi id không tồn tại.

Giải thích: Status là tín hiệu máy đọc được để client quyết định flow.

Lỗi/hiểu sai thường gặp: Luôn trả 200 và nhét lỗi vào body.

Best practice: Phân biệt success/client error/server error.

Tự kiểm tra: Giải thích 401 vs 403 bằng một tình huống.

II.7 Client–Server

Mục tiêu: Hiểu separation of concerns.

Ví dụ: React lo UI; Spring Boot lo business/data.

Giải thích: Tách vai trò cho phép client/server tiến hóa độc lập dựa trên contract.

Lỗi/hiểu sai thường gặp: Nhét business rule quan trọng hoàn toàn phía client.

Best practice: Xác định rõ responsibility và contract boundary.

Tự kiểm tra: Phân loại 10 trách nhiệm vào client hoặc server.

II.8 Statelessness

Mục tiêu: Hiểu mỗi request phải độc lập về context.

Ví dụ: Request bảo mật mang credential/token cần thiết thay vì server dựa vào “request trước”.

Giải thích: Stateless tăng khả năng scale và giảm coupling với session state.

Lỗi/hiểu sai thường gặp: Nghĩ stateless = không có authentication hoặc không có database.

Best practice: Tách resource state khỏi client session state.

Tự kiểm tra: Nêu 2 lợi ích và 1 trade-off của statelessness.

II.9 Cacheability

Mục tiêu: Hiểu response có thể tuyên bố khả năng cache.

Ví dụ: GET danh mục ít thay đổi có thể được cache theo policy phù hợp.

Giải thích: Cache giảm latency/load nhưng phải kiểm soát freshness.

Lỗi/hiểu sai thường gặp: Cache mọi response hoặc cache dữ liệu nhạy cảm không kiểm soát.

Best practice: Cache theo semantics và freshness requirement.

Tự kiểm tra: Chỉ ra endpoint FUNews nào phù hợp/không phù hợp để cache.

II.10 Uniform Interface

Mục tiêu: Hiểu giao diện thống nhất giảm coupling.

Ví dụ: Cùng pattern GET collection, GET item, POST collection cho nhiều resource.

Giải thích: Client có thể dự đoán cách API hoạt động nhờ naming/method/status nhất quán.

Lỗi/hiểu sai thường gặp: Mỗi team/resource đặt quy tắc riêng.

Best practice: Có conventions/API style guide.

Tự kiểm tra: So sánh hai API nhất quán và không nhất quán.

II.11 Layered System

Mục tiêu: Hiểu client không cần biết toàn bộ topology server.

Ví dụ: Client gọi API qua gateway/load balancer nhưng vẫn theo contract.

Giải thích: Layering hỗ trợ security, scale, observability.

Lỗi/hiểu sai thường gặp: Cho rằng layered = bắt buộc microservices.

Best practice: Layering là constraint kiến trúc; số lớp tùy hệ thống.

Tự kiểm tra: Vẽ một flow Client → Gateway → Service → DB.

II.12 Sub-resource

Mục tiêu: Biểu diễn quan hệ parent-child khi có ý nghĩa nghiệp vụ.

Ví dụ: GET /api/news/10/comments.

Giải thích: Sub-resource làm rõ context của child trong parent.

Lỗi/hiểu sai thường gặp: Lồng URI quá sâu cho mọi quan hệ.

Best practice: Chỉ lồng khi quan hệ/context thực sự quan trọng và depth còn dễ hiểu.

Tự kiểm tra: Thiết kế tags/comments của News theo 2 phương án và so sánh.

II.13 REST API design best practices

Mục tiêu: Dùng nouns, consistency, versioning/error/documentation awareness.

Ví dụ: /api/v1/news-articles.

Giải thích: Best practice giúp API dễ đoán, dễ tích hợp và tiến hóa.

Lỗi/hiểu sai thường gặp: Versioning mọi thay đổi nhỏ hoặc bỏ qua documentation.

Best practice: Thiết kế contract trước code; document quyết định.

Tự kiểm tra: Review một API contract theo 6 tiêu chí.

II.14 Benefits of RESTful APIs

Mục tiêu: Liên hệ scalability/flexibility/interoperability/simplicity/evolution với sản phẩm.

Ví dụ: Một News API phục vụ React web và mobile client.

Giải thích: Tách giao diện giúp nhiều client dùng chung backend.

Lỗi/hiểu sai thường gặp: Xem REST là “nhanh hơn” trong mọi trường hợp.

Best practice: Đánh giá lợi ích theo context, không khẩu hiệu.

Tự kiểm tra: Chọn 3 lợi ích quan trọng nhất cho FUNews và giải thích.

II.15 Spring Boot overview

Mục tiêu: Hiểu Boot giúp khởi tạo/chạy ứng dụng Spring dễ hơn.

Ví dụ: Starter Spring Web + embedded server.

Giải thích: Boot cung cấp convention, auto-configuration, starters và runtime thuận tiện.

Lỗi/hiểu sai thường gặp: Nghĩ Spring Boot = Spring Framework = REST.

Best practice: Phân biệt framework nền tảng, Boot và REST design.

Tự kiểm tra: Giải thích embedded server và starter dependency.

II.16 Vì sao Spring Boot phù hợp cho REST

Mục tiêu: Liên hệ rapid development, maintainability, production features, ecosystem.

Ví dụ: Tạo skeleton từ Initializr với Spring Web.

Giải thích: Boot giảm setup để tập trung vào API/business.

Lỗi/hiểu sai thường gặp: Dùng annotation mà không hiểu contract.

Best practice: Design first, framework second.

Tự kiểm tra: Nêu 3 lý do chọn Boot cho bài tập backend môn học.

II.17 Kỹ năng đầu ra & hiểu sai phải tránh

Mục tiêu: Tổng hợp khả năng thiết kế và giải thích.

Ví dụ: Từ yêu cầu “quản lý News” thiết kế được contract cơ bản.

Giải thích: Sinh viên đạt khi giải thích được quyết định chứ không chỉ chép endpoint.

Lỗi/hiểu sai thường gặp: Nhầm REST/Spring Boot, endpoint/action, 401/403, stateless/session.

Best practice: Luôn trace từ requirement → resource → API contract.

Tự kiểm tra: Tự thiết kế API cho Course hoặc Product mà không nhìn mẫu.

II.18 REST Constraint Trace – bảng thực hành nhanh

Constraint	Câu hỏi kiểm tra	Ví dụ FUNews
Client–Server	UI và business/data đã tách chưa?	React admin vs Spring Boot backend
Stateless	Request có đủ context để xử lý độc lập?	Authorization context đi cùng request
Cacheable	Response nào có thể cache và trong bao lâu?	Category list có thể ổn định hơn News admin
Uniform Interface	Naming/method/status có nhất quán?	/api/news, /api/categories cùng pattern
Layered System	Client có phụ thuộc topology nội bộ không?	Client vẫn gọi contract dù qua gateway

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

III.1 REST trong các loại hệ thống

Hệ thống	Resource tiêu biểu	Interaction REST
E-commerce	products, orders, customers	GET /api/products; POST /api/orders
Banking	accounts, transactions	GET balance; POST transaction với bảo mật chặt
Social network	posts, comments, users	GET/POST posts; nested comments
Learning management	courses, students, enrollments	GET courses; GET students
News management	news, categories, users, tags	CRUD News/Category/User

III.2 Case study xuyên suốt: FUNewsManagementSystem

Domain từ Assignment 01 được tái sử dụng để sinh viên thấy một hệ thống phát triển liên tục: frontend mock → API contract → Spring Boot backend → persistence → tích hợp React.

Resource	Collection URI	Item URI	Quan hệ/ghi chú
News Article	/api/news	/api/news/{id}	category, author, tags, status
Category	/api/categories	/api/categories/{id}	active/inactive
Account/User	/api/users	/api/users/{id}	role Admin/Staff
Tag	/api/tags	/api/tags/{id}	có thể liên kết nhiều News

III.3 Frontend action → API contract trace

Frontend behavior	API candidate	Success	Failure cần nghĩ tới
Mở News Management	GET /api/news	200 + JSON list	500; empty list không phải lỗi
Mở Edit #10	GET /api/news/10	200 + News	404 nếu không tồn tại
Create News	POST /api/news	201 + created representation	400 validation; 401/403 auth
Update News #10	PUT /api/news/10	200/204 tùy contract	400/404/403
Delete News #10	DELETE /api/news/10	200/204 tùy contract	404/403/conflict theo nghiệp vụ
Search title	GET /api/news?keyword=react	200 + filtered list	400 nếu query invalid theo contract

III.4 API Contract Card – tối thiểu trước khi code

Field	Ví dụ
Use case	Admin tạo một News Article
Method	POST
URI	/api/news
Request representation	JSON: title, content, categoryId, status, tags...
Success status	201 Created
Success response	News vừa tạo / hoặc location theo contract
Possible errors	400, 401, 403
Consumer	React Assignment 01 / future client

III.5 Tại sao API contract giúp teamwork

- Frontend có thể mock response trước khi backend xong.
- Backend biết chính xác input/output cần cung cấp.
- Tester viết test cases dựa trên status/representation.
- Khi thay đổi contract, team biết phạm vi ảnh hưởng.

- OpenAPI/Swagger ở chapter sau có thể formalize contract này.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN THỰC HIỆN VÀ TRIỂN KHAI

Phần IV không xây CRUD Spring Boot. Mục tiêu là tạo năng lực thiết kế REST và chuẩn bị skeleton/tooling để Part B bắt đầu từ một contract đã được kiểm chứng.

IV.A. REST LÀ GÌ?

Mục tiêu: sinh viên có thể định nghĩa REST và architectural style.

- Định nghĩa REST và architectural style.
- Client–Server, Statelessness, Cacheability, Uniform Interface.
- Ví dụ React client ↔ Spring Boot server.
- Phân biệt REST với framework/library.

TRACE

```
Requirement: "quản lý tin tức"
→ Resource: news
→ Collection URI: /api/news
→ Item URI: /api/news/{id}
→ Interactions: GET/POST/PUT/DELETE
→ Representation: JSON
→ Outcomes: HTTP status codes
```

Checkpoint — Nếu sinh viên bắt đầu bằng câu “em sẽ tạo @RestController...” nhưng chưa có resource/URI/method/status, quay lại bước thiết kế.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.B. RESTFUL WEB SERVICES

Mục tiêu: sinh viên có thể HTTP làm kênh giao tiếp.

- HTTP làm kênh giao tiếp.
- Resource + URI + HTTP verbs + representation.
- Ví dụ Product/News/Category.

HTTP EXAMPLE

```
GET /api/news/10 HTTP/1.1
Host: localhost:8080
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "title": "React and Spring Boot",
  "status": 1,
  "categoryId": 3
}
```

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.C. ELEMENTS OF RESTFUL APIs

Mục tiêu: sinh viên có thể resource discovery.

- Resource discovery.
- HTTP method mapping.
- Status-code planning.
- Representation sketch.

Hành động	Method	Endpoint	Success
Lấy list	GET	/api/news	200
Lấy item	GET	/api/news/{id}	200 / 404
Tạo mới	POST	/api/news	201
Cập nhật	PUT	/api/news/{id}	200/204
Xóa	DELETE	/api/news/{id}	200/204

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.D. RESTFUL API DESIGN BEST PRACTICES

Mục tiêu: sinh viên có thể nouns, simple/consistent URL.

- Nouns, simple/consistent URL.
- HTTP method semantics.
- Versioning awareness.
- Error handling awareness.
- Documentation/OpenAPI awareness.

Chưa tốt	Tốt hơn	Lý do
GET /api/getAllNews	GET /api/news	Method đã nói “get”; URI chỉ nên nói resource
POST /api/createNews	POST /api/news	POST biểu thị create
GET /api/deleteNews?id=10	DELETE /api/news/10	Method + item URI rõ intent
/api/news_articles	/api/news-articles	Giữ naming convention nhất quán

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.E. BENEFITS OF RESTFUL APIs

Mục tiêu: sinh viên có thể scalability.

- Scalability.
- Flexibility.
- Interoperability.
- Simplicity.
- Evolution.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.F. REST ARCHITECTURE

Mục tiêu: sinh viên có thể client–Server.

- Client–Server.
- Stateless.
- Cacheable.
- Uniform Interface.
- Layered System.

Constraint	Câu hỏi workshop
Client–Server	Logic này thuộc UI hay backend?
Stateless	Request này có phụ thuộc request trước không?
Cacheable	Client/proxy có được phép reuse response không?
Uniform Interface	Người mới có đoán được endpoint/method không?
Layered System	Client có cần biết service nội bộ/gateway không?

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.G. URI VÀ SUB-RESOURCE

Mục tiêu: sinh viên có thể collection URI vs item URI.

- Collection URI vs item URI.
- Sub-resource cho quan hệ.
- Tránh nesting quá sâu.

URI EXAMPLES

```

/api/news                # collection
/api/news/10            # item
/api/categories/5/news   # news trong category 5 (nếu contract chọn nested)
/api/news/10/comments   # comments của news 10
/api/news/10/comments/5 # comment 5 trong context news 10

```

Bài tập nhỏ: với quan hệ News–Tag, hãy đề xuất 2 thiết kế URI (nested và non-nested), sau đó giải thích ưu/nhược điểm.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.H. HTTP METHODS

Mục tiêu: sinh viên có thể GET/POST/PUT/DELETE ở mức tổng quan.

- GET/POST/PUT/DELETE ở mức tổng quan.
- CRUD mapping.
- Không đi sâu request-body/annotation.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.I. HTTP STATUS CODES

Mục tiêu: sinh viên có thể 200, 201, 400, 401, 403, 404, 500.

- 200, 201, 400, 401, 403, 404, 500.
- Status code phải gắn với outcome.

Status	Tình huống FUNews
200 OK	GET list/item thành công
201 Created	POST tạo News thành công
400 Bad Request	Payload thiếu/không hợp lệ theo contract
401 Unauthorized	Chưa có credential hợp lệ
403 Forbidden	Đã xác thực nhưng không có quyền
404 Not Found	News id không tồn tại
500 Internal Server Error	Lỗi server không dự kiến

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.J. SPRING BOOT OVERVIEW

Mục tiêu: sinh viên có thể auto-configuration.

- Auto-configuration.
- Starter dependencies.
- Embedded server.
- Production-ready features.

ENVIRONMENT PREPARATION

```
Spring Initializr
Project: Maven
Language: Java
Spring Boot: course-approved 3.x
Java: 21
Dependency: Spring Web

↓
Generate → Open in IntelliJ → Maven sync → Run skeleton
```

Mục tiêu thao tác — Chỉ xác nhận project skeleton chạy được. Không tạo controller CRUD ở Part A.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.K. WHY SPRING BOOT FOR RESTFUL APIs?

Mục tiêu: sinh viên có thể simplified development.

- Simplified development.
- Embedded runtime.
- Ecosystem/microservices support.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.L. BENEFITS OF SPRING BOOT FOR REST

Mục tiêu: sinh viên có thể rapid development.

- Rapid development.
- Productivity.
- Maintainability.
- Scalability/reliability.

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.M. SPRING BOOT VS OTHER FRAMEWORKS

Mục tiêu: sinh viên có thể so sánh ở mức khái niệm, không xếp hạng.

- So sánh ở mức khái niệm, không xếp hạng.
- Tập trung setup/dependency/runtime/ecosystem.

Tiêu chí	Spring Boot	Một framework/backend stack khác
Setup	Convention + auto-configuration	Có thể cần cấu hình theo stack
Server runtime	Embedded server phổ biến	Tùy framework/runtime
Dependency	Starter ecosystem	Cơ chế package/dependency riêng
Enterprise ecosystem	Spring ecosystem rộng	Khác nhau theo nền tảng
Mục tiêu môn học	Phù hợp Java/Spring learning path	Không phải baseline SBA301

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.N. PREVIEW PART B

Mục tiêu: sinh viên có thể annotations + controller/service/repository.

- Annotations + controller/service/repository.
- Request/response handling.
- Chỉ preview, không thực thi CRUD.

JAVA PREVIEW — KHÔNG YÊU CẦU TRIỂN KHAI

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    // Preview only: Part B sẽ triển khai annotations và CRUD chi tiết
}
```

Lỗi/hiểu sai cần tránh: biến phần này thành bài thuộc Part B; học annotation trước khi hiểu contract; sao chép endpoint mà không giải thích được resource/method/status.

Best practice: mọi quyết định phải trace được về requirement hoặc REST semantics.

Bài tập nhỏ: viết 1 câu hỏi “vì sao?” cho chính thiết kế của bạn và tự trả lời bằng evidence/semantics.

IV.15 Workshop 1 – Resource Discovery Worksheet

Requirement phrase	Candidate resource	Không nên là resource vì...	Ghi chú
Manage news article	news	—	Core domain resource

Search news	news	search là action/query, không nhất thiết resource	Dùng query parameter theo contract
Login	?	Cần phân tích auth design riêng	Không biến mọi action thành CRUD resource
Manage categories	categories	—	Core resource
Manage accounts	users/accounts	—	Chọn naming nhất quán

IV.16 Workshop 2 – API Contract Worksheet

Use case	Method	URI	Request	Success response/status	Errors
List News					
Get News by id					
Create News					
Update News					
Delete News					
Search News					

IV.17 Workshop 3 – Postman Observation Sheet

Quan sát	Giá trị
Request method	
Request URL	
Request headers	
Status code	
Response Content-Type	
Response body type	
Resource represented	
Điều gì chứng minh đây là interaction HTTP?	

IV.18 Chuẩn bị Spring Boot skeleton trong IntelliJ

1. Bước 1: Mở Spring Initializr và chọn Maven, Java, JDK 21.
2. Bước 2: Chọn Spring Boot 3.x theo baseline môn học và dependency Spring Web.
3. Bước 3: Generate project, giải nén và Open bằng IntelliJ.
4. Bước 4: Chờ Maven sync; kiểm tra JDK Project SDK = 21.
5. Bước 5: Run application skeleton.
6. Bước 6: Quan sát console: embedded server khởi động thành công.
7. Bước 7: Dừng application và commit checkpoint nếu dùng Git.

Verification — Checkpoint đạt khi application skeleton khởi động/dừng được, Maven không lỗi và sinh viên giải thích được Spring Web/embedded server. Không cần endpoint CRUD.

IV.19 Kế hoạch lớp 135 phút gợi ý

Thời gian	Hoạt động	Evidence
0–10	Retrieval Slot 8/9 + 3–2–1	5 câu retrieval
10–30	REST/resource/URI mental model	Resource worksheet
30–50	Methods/status + endpoint critique	API design corrections
50–70	REST constraints peer teaching	Constraint trace
70–90	FUNews API contract workshop	News/Category/User contract
90–105	Postman observation	Request/response evidence

SBA301 • Slot 12 • Chapter 10 – RESTful Web Services • Part A

105–120	Spring Boot overview + Initializr skeleton	Project runs
120–130	Break-It REST design challenge	Fix anti-patterns
130–135	Exit ticket + bridge Part B	3 answers + 1 question

V. KINH NGHIỆM THỰC TIỄN, LỖI THƯỜNG GẶP VÀ TIPS

V.1 Anti-pattern → chẩn đoán → sửa

Anti-pattern	Vì sao có vấn đề	Cách sửa
/api/getAllNews	URI chứa action đã có trong GET	GET /api/news
/api/deleteNews?id=10	GET/query thể hiện xóa sai semantics	DELETE /api/news/10
POST cho mọi thao tác	Mất semantics, khó cache/debug/document	Mapping method đúng intent
Luôn trả 200	Client khó phân biệt outcomes	Dùng status code phù hợp
URI lồng 5–6 cấp	Coupling và khó đọc	Rút gọn, cân nhắc query/non-nested resource
Thiết kế từ tên controller method	Framework-driven thay vì domain-driven	Bắt đầu từ use case/resource
Spring Boot = REST	Nhầm tool với architecture	Tách design layer và implementation framework
Stateless = không auth	Hiểu sai constraint	Auth context vẫn có thể gửi mỗi request

V.2 REST design decision checklist

- Resource có phải danh từ/domain concept không?
- Collection/item URI có nhất quán không?
- HTTP method có đúng intent không?
- Success status hợp lý chưa?
- Failure states nào cần contract?
- Representation có expose field không cần thiết/nhạy cảm không?
- Sub-resource có thực sự giúp thể hiện context?
- Frontend có thể đoán API từ pattern không?
- Contract có đủ để viết Postman test không?

V.3 Debug API design trước khi debug code

Khi Part B bắt đầu, nhiều lỗi tưởng là “Spring lỗi” thực chất là contract chưa rõ. Quy trình nên là: Requirement → Contract → HTTP trace → Controller mapping → Service logic → Persistence. Part A chịu trách nhiệm làm chắc hai bước đầu.

V.4 Break-It Lab

Bug thiết kế	Nhiệm vụ sinh viên
GET /api/news/delete/10	Tìm ít nhất 2 vi phạm semantics và sửa
POST /api/news/10/getComments	Thiết kế lại bằng sub-resource
GET /api/users/999 → 200 {error:"not found"}	Đề xuất status/response tốt hơn
/api/News_Articles	Chuẩn hóa naming convention
Mỗi request phụ thuộc server-side flag từ request trước	Phân tích vấn đề statelessness

VI. BÀI TẬP CHỦ ĐỀ & BÀI TẬP TỔNG HỢP

VI.A Bài tập chủ đề

Bài 1 – Xác định resource

Từ FUNews, phân loại News, Category, User, Tag, Login, Search: đâu là resource, đâu là action/query/use case.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 2 – Thiết kế URI

Thiết kế collection/item URI cho News, Category, User; giải thích naming convention.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 3 – CRUD ↔ HTTP method

Mapping 10 thao tác sang GET/POST/PUT/DELETE và giải thích.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 4 – Status code

Chọn 200/201/400/401/403/404/500 cho 10 tình huống.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 5 – Endpoint critique

Sửa /getAllNews, /createNews, /updateNewsById, /deleteNews.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 6 – Sub-resource

Thiết kế comments/tags của một News; giải thích khi nào không nên nested.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 7 – Postman observation

Gọi JSONPlaceholder posts và ghi evidence method/status/header/body.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

Bài 8 – Spring Initializr

Tạo Spring Boot 3.x + Java 21 + Spring Web skeleton và chạy trong IntelliJ.

Deliverable: 3–8 dòng giải thích + bảng/ảnh evidence nếu áp dụng.

VI.B Bài tập tổng hợp – REST API Design for FUNewsManagementSystem

Mục tiêu: tạo một API design package đủ rõ để Part B có thể triển khai mà không phải “đoán” requirement.

Deliverable	Yêu cầu tối thiểu
1. Resource catalog	News Article, Category, Account/User, Tag (nếu dùng)
2. API table – News	GET list/item, POST, PUT, DELETE, Search
3. API table – Category	GET list/item, POST, PUT, DELETE
4. API table – User	GET list/item, POST, PUT, DELETE; role/status awareness
5. Sub-resource	Ít nhất một quan hệ nếu phù hợp
6. Status matrix	Success + 400/401/403/404/500
7. Representations	Ít nhất 1 request JSON + 1 response JSON + 1 error representation
8. Rationale	Giải thích 5 design decisions
9. README/API contract	Trình bày ngắn gọn, nhất quán
10. Peer review	Một bạn khác review theo REST checklist

Mẫu JSON để phân tích representation (không phải code Spring Boot)

JSON REPRESENTATION EXAMPLE

Phucpt10@fpt.edu.vn

SBA301 – REST Fundamentals with Spring Boot

Trang 20

```
{  
  "id": 10,  
  "title": "REST Design Fundamentals",  
  "content": "...",  
  "categoryId": 2,  
  "status": 1,  
  "tags": ["REST", "Spring"]  
}
```

Quality Gate trước khi chuyển Part B

- Không còn endpoint bắt đầu bằng get/create/update/delete cho CRUD cơ bản.
- Mỗi use case có method + URI + success status.
- Có phân biệt 401/403/404.
- Resource/URI naming nhất quán.
- Có ít nhất một explanation cho stateless/cacheable/uniform interface.
- Spring Boot skeleton chạy được nhưng chưa có CRUD implementation.
- Sinh viên có thể trình bày API contract trong 3 phút không nhìn tài liệu.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ trong bài
REST	Phong cách kiến trúc cho hệ thống phân tán qua network.	Thiết kế API cho FUNews
RESTful Web Service	Web service tuân theo REST principles/constraints.	News API
Resource	Thực thể/khái niệm domain được expose.	news, categories
URI	Định danh resource.	/api/news/10
URL	Một loại URI có vị trí/truy cập resource.	http://localhost:8080/api/news
Endpoint	Điểm giao tiếp API, thường hiểu method + URI.	GET /api/news
Representation	Dữ liệu biểu diễn trạng thái resource.	JSON News
JSON	Định dạng text phổ biến cho representation.	{"id":10,...}
XML	Representation format khác có thể dùng.	<news>...</news>
GET	Method đọc resource.	GET /api/news
POST	Method thường dùng tạo resource.	POST /api/news
PUT	Method thường dùng cập nhật resource.	PUT /api/news/10
DELETE	Method xóa resource.	DELETE /api/news/10
HTTP Status Code	Mã kết quả HTTP.	200/201/404
200 OK	Request thành công.	GET thành công
201 Created	Resource được tạo.	POST thành công
400 Bad Request	Request không hợp lệ.	Payload thiếu field
401 Unauthorized	Chưa xác thực/credential không hợp lệ.	API bảo mật
403 Forbidden	Đã xác thực nhưng không đủ quyền.	Staff xóa User
404 Not Found	Resource không tồn tại.	News id 999
500 Internal Server Error	Lỗi server không dự kiến.	Exception server
Client–Server	Tách UI/client và server/data/business.	React ↔ Spring Boot
Stateless	Request độc lập về context.	Token/context mỗi request
Cacheable	Response có thể khai báo khả năng cache.	GET categories
Uniform Interface	Giao diện tương tác thống nhất.	Pattern method/URI/status
Layered System	Cho phép nhiều lớp trung gian.	Gateway → service
Sub-resource	Resource trong context resource cha.	/news/10/comments
API Versioning	Quản lý phiên bản contract.	/api/v1/news
API Contract	Thỏa thuận method/URI/request/response/status.	News API table
Spring Boot	Nền tảng tạo ứng dụng Spring nhanh.	Backend SBA301
Spring Web	Starter/module cho web/HTTP trong Spring ecosystem.	Dependency project
Embedded Server	Server đóng gói/chạy cùng ứng dụng.	Tomcat embedded
Auto-configuration	Boot tự cấu hình dựa trên dependencies/context.	Spring Web setup
Starter Dependency	Gói dependency theo use case.	spring-boot-starter-web
Postman	Công cụ gửi/quan sát HTTP requests.	Test API
OpenAPI/Swagger	Chuẩn/công cụ mô tả API.	Chapter 11
3-layer architecture	Controller–Service–Repository ở mức preview.	Part B

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

Mục tiêu của AI trong Slot 12 là tăng chất lượng reasoning về API design, không thay thế việc sinh viên tự quyết định contract. Quy tắc: AI may generate; student must verify bằng HTTP semantics, REST constraints và requirement.

VIII.1 10 prompt nghiên cứu chuyên sâu kỹ thuật

8. 1. Phân tích từng REST constraint và cho ví dụ sai/đúng với FUNewsManagementSystem.
9. 2. Giải thích statelessness ảnh hưởng đến authentication và horizontal scaling như thế nào.
10. 3. So sánh REST, RPC và SOAP ở mức architectural interaction.
11. 4. Phân biệt URI, URL và URN với ví dụ liên quan API.
12. 5. Phân tích cacheability và các loại response không nên cache.
13. 6. Giải thích uniform interface làm giảm coupling giữa React và Spring Boot như thế nào.
14. 7. Phân tích layered system với API gateway/load balancer/service.
15. 8. Giải thích idempotency của GET, PUT, DELETE và vì sao POST thường khác.
16. 9. So sánh JSON và XML như representation, không đồng nhất chúng với REST.
17. 10. Phân tích ưu/nhược điểm nouns số nhiều và naming conventions trong REST API.

VIII.2 10 prompt thực tiễn chuyên nghiệp

18. 1. Tạo checklist review API contract cho frontend/backend team.
19. 2. Phân tích khi nào dùng path parameter, query parameter và request body.
20. 3. Đề xuất strategy versioning cho API đang có nhiều client.
21. 4. Đề xuất error response convention nhất quán cho validation/not-found/authorization.
22. 5. Thiết kế pagination/sorting/filtering contract cho GET /api/news ở mức khái niệm.
23. 6. Phân tích cách OpenAPI giúp contract-first collaboration.
24. 7. Thiết kế resource boundary cho News–Category–Tag để tránh endpoint quá lồng.
25. 8. Phân tích security concerns cần chuẩn bị nhưng chưa triển khai trong Part A.
26. 9. Đề xuất Postman test matrix từ API contract FUNews.
27. 10. Phân tích cách đổi từ mock data Assignment 01 sang Spring Boot API mà giảm thay đổi frontend.

VIII.3 10 prompt khám phá công nghệ/xu hướng

28. 1. So sánh REST, GraphQL, gRPC và tRPC theo kiểu client, contract, tooling và use case.
29. 2. HTTP/2 và HTTP/3 thay đổi transport nhưng không thay thế REST semantics như thế nào?
30. 3. OpenAPI 3 được dùng trong contract-first development ra sao?
31. 4. Spring HATEOAS là gì và liên hệ với REST maturity như thế nào?
32. 5. API Gateway hỗ trợ routing/security/observability ra sao?
33. 6. Rate limiting và throttling bảo vệ API production như thế nào?
34. 7. Observability cho REST API gồm logs, metrics, traces như thế nào?
35. 8. API governance là gì trong tổ chức nhiều team?
36. 9. AI có thể hỗ trợ review OpenAPI/REST contract nhưng rủi ro hallucination nào cần kiểm soát?
37. 10. Khi nào một hệ thống nên chọn không dùng REST cho một interaction?

VIII.4 AI Verification Worksheet

AI claim/gợi ý	Nguồn kiểm chứng	Test/Reasoning	Quyết định cuối
Endpoint /api/getNews tốt hơn /api/news?	REST resource naming	Method đã mang intent GET	Reject
201 hay 200 cho create?	HTTP semantics + contract	Xem requirement/response	
Nested URI cho tags?	Domain relationship	So sánh depth/query/non-nested	

VIII.5 AI-0 Human Gate

- Không dùng AI: tự thiết kế 5 endpoint News trong 5 phút.
- Giải thích 3 endpoint bằng method/URI/status semantics.
- Sau đó mới dùng AI review; đánh dấu ít nhất một gợi ý AI được chấp nhận hoặc bác bỏ và lý do.
- Không coi “AI nói vậy” là evidence.

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

IX.1 Kiến thức nền tảng

- REST là gì và không phải là gì?
- RESTful Web Service khác REST như thế nào?
- Resource khác representation như thế nào?
- Collection URI và item URI khác nhau ra sao?
- GET/POST/PUT/DELETE dùng cho intent nào?
- 200 và 201 khác nhau khi nào?
- 401 và 403 khác nhau thế nào?
- Stateless không đồng nghĩa điều gì?
- Cacheable mang lại lợi ích và rủi ro gì?
- Uniform Interface giúp client/server như thế nào?
- Layered System nghĩa là gì?
- Sub-resource nên dùng khi nào?
- Spring Boot giúp gì cho REST development?
- Vì sao Part A chưa cần annotations?

IX.2 Kỹ năng thực hành

Tôi có thể...	Đạt	Chưa đạt	Cần làm lại
Xác định resource từ requirement			
Thiết kế collection/item URI			
Mapping CRUD với method			
Chọn status cơ bản			
Thiết kế sub-resource hợp lý			
Đọc HTTP request/response Postman			
Viết API contract table			
Giải thích 5 REST constraints trong phạm vi bài			
Tạo Spring Boot skeleton Java 21 + Spring Web			
Giải thích vì sao Spring Boot phù hợp			

IX.3 Sản phẩm cần có sau buổi học

- Resource catalog cho FUNews hoặc domain tương đương.
- API contract fit nhất cho một resource đầy đủ CRUD.
- Một JSON representation mẫu.
- Postman observation evidence.
- Spring Boot skeleton chạy được trong IntelliJ.
- Một reflection: “design decision khó nhất của tôi là gì và vì sao?”

IX.4 Mức tự đánh giá

Mức	Dấu hiệu
Mức 1 – Chưa đạt	Không phân biệt resource/endpoint; mapping method/status còn đoán.
Mức 2 – Đạt cơ bản	Thiết kế CRUD endpoint nhất quán và tạo được Spring Boot skeleton.
Mức 3 – Tốt	Giải thích REST constraints, sub-resource, trade-off và thiết kế FUNews contract rõ ràng.

IX.5 Exit Ticket

1. Viết một endpoint tốt và một endpoint chưa tốt cho thao tác lấy danh sách News; giải thích.
2. Giải thích 401 vs 403 bằng FUNews role Admin/Staff.
3. Một câu: Stateless giúp scale như thế nào?

- 4. Part B sẽ biến thành phần nào của contract thành annotation/controller?

IX.6 Bridge sang Chapter 10 Part B

NEXT-SESSION BRIDGE

Part A

Requirement → Resource → URI → Method → Status → Representation → API Contract

↓

Part B

@RestController / mapping annotations

↓

Controller → Service → Repository

↓

Request/Response handling

↓

CRUD API chạy thật

Kết luận — Nếu sinh viên chưa tự thiết kế được API contract và giải thích REST semantics, chưa nên chuyển trọng tâm sang annotations. Framework code chỉ có ý nghĩa khi contract đã rõ.

PHỤ LỤC A – QUICK REFERENCE

Mục tiêu	Pattern
Collection	/api/resources
Item	/api/resources/{id}
List	GET collection
Create	POST collection
Read one	GET item
Update	PUT item
Delete	DELETE item
Success read	200
Success create	201
Bad input	400
Unauthenticated	401
Forbidden	403
Not found	404
Server failure	500

REST design mantra

Resource là danh từ. URI nói “cái gì”. HTTP method nói “làm gì”. Status code nói “kết quả gì”. Representation nói “trạng thái được truyền như thế nào”.

PHỤ LỤC B – EVIDENCE PACKAGE

- E1 – Pre-class 3–2–1 Learning Note.
- E2 – Resource Discovery Worksheet.
- E3 – REST Constraint Trace.
- E4 – FUNews API Contract.
- E5 – Endpoint anti-pattern corrections.
- E6 – Postman observation screenshot/notes.
- E7 – Spring Initializr configuration screenshot.
- E8 – IntelliJ console showing skeleton runs.
- E9 – AI Verification Worksheet (nếu dùng AI).
- E10 – Exit Ticket + reflection.

PHỤ LỤC C – INSTRUCTOR / PEER CHECK QUESTIONS

1. Bạn chọn News là resource dựa trên requirement nào?
2. Tại sao endpoint không nên là /getAllNews?
3. Nếu GET /api/news/999 không có dữ liệu, vì sao 404 hợp lý hơn 200 + error string?
4. 401 và 403 khác nhau thế nào trong role Admin/Staff?
5. Stateless có cấm database không?
6. Cacheable có nghĩa mọi response phải cache không?
7. Khi nào nested resource làm API khó bảo trì?
8. Nếu React route là /news/10 thì backend endpoint có bắt buộc giống hệt không?
9. Vì sao cần API contract trước controller?
10. Part B sẽ thay đổi điều gì nhưng giữ nguyên điều gì từ Part A?